



Universidad autónoma de baja california

Ingeniería en computación

Internet de las cosas

Lab-taller 10: node-red

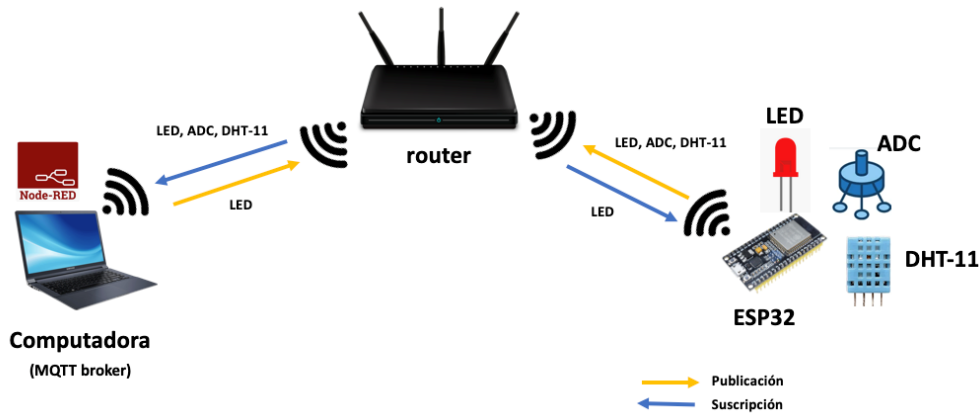
Maestro: Dr. Aguilar Noriega Leocundo

Garcia Chaves erik

Viernes 20 de marzo del 2026

1. Instrucciones

El objetivo es mediante el protocolo MQTT publicar datos de dispositivos eléctricos al bróker y poder visualizarlos mediante la herramienta **node-red**



- se integrará a un ESP32 un led, un ADC y el sensor de temperatura y humedad DHT-11
- se realizará un programa que publique en el bróker los estados del LED, del ADC, y los valores de la temperatura y humedad que esta detectando el sensor, además que este suscrito a un tópico de LED el cual funcionara para poder prender y apagar el led desde **node-red**
- realice un flujo en **node-red** para que se pueda suscribir y publicar en el bróker con el fin de
 - visualizar el estado del led **ON/OFF**
 - visualizar el estado del ADC (**volts**)
 - visualizar el estado actual de la humedad relativa (%)
 - visualizar el valor actual de la temperatura (**C**)
 - encender o apagar led mediante un botón.

2. Desarrollo

el Desarrollo con **node-red** el cual es una herramienta visual que ayua a conectar APIs, servicios en línea y productos IoT.

En este caso se utiliza la comunicación **MQTT** para publicar y suscribirse a los distintos tópicos que necesita este proyecto. El código de este programa es muy parecido a una publicación y suscripción MQTT típica entre 2 o más dispositivos, salvo que ahora se requiere un tercero más que estará suscrito o publicará en los mismos tópicos.

Como primero deberíamos de establecer los tópicos que el proyecto va a utilizar los cuales serían:

```

1  #define BROKER "mqtt://192.168.1.66:1883"
2  #define TOPIC_LED "device/led"
3  #define TOPIC_ADC "device/adc"
4  #define TOPIC_HUM "device/humi"
5  #define TOPIC_TEMP "device/temp"
6  //este es para activar el led el cual lo hace desde el broker
7
8  #define TOPIC_ACT "device/action"
9

```

El ultimo seria de acción, para poder encender el led desde la interfaz gráfica de **node-red** así como la modificación de la dirección IP del bróker mosquitto, porque ahora estará corriendo localmente.

Lo primero que hará MQTT cuando el ESP se pueda conectar a wifi es suscribirse a ese tópico

```

switch ((esp_mqtt_event_id_t)event_id){
    case MQTT_EVENT_CONNECTED: {
        ESP_LOGI(TAG, "MQTT_EVENT_CONNECTED");

        //por medio de un evento indicamos que se conecto
        esp_mqtt_client_subscribe(client, TOPIC_ACT, 0);
        ESP_LOGI(TAG, "Subscribed to topic %s", TOPIC_ACT);
    }
}

```

El evento que se activa cuando se reciben los datos de cuando se publica en el tópico que el ESP está suscrito, se toma esa información y lo guardamos en buffers para poder operar sobre esa información, utilizamos una cola que servirá para enviar la información desde este case en el archivo **mqtt_lib.c** hacia el **main.c** a la función **task_input_topic()** que es la que procesara lo que se recibió y decidirá qué acción es la que se debe de tomar.

```

case MQTT_EVENT_DATA:{
    //en este punto recibi el dato desde el servidor esto para poder actualizar el estado del led
    char topic[128] = {0};
    char data[256] = {0};

    snprintf(topic, sizeof(topic), "%.*s", event->topic_len, event->topic);
    snprintf(data, sizeof(data), "%.*s", event->data_len, event->data);

    ESP_LOGI(TAG, "Topic : %s", topic);
    ESP_LOGI(TAG, "Data : %s", data);

    uint16_t length = event->topic_len + event->data_len;
    char *msg = malloc(length + 2); // +1 para ':' y +1 para '\0'
    if (!msg) {
        ESP_LOGE(TAG, "No memory for msg");
        return;
    }
    snprintf(msg, length + 2, "%s:%s", topic, data);

    //vamos a mandar por medio de la cola
    xQueueSend(flow_data, &msg, portMAX_DELAY);
}

```

El sistema utiliza 3 GPIOs 1 para el led, 1 que prende o apaga el led y otro que es el botón que deja mandar la información, realiza la acción de **publicar** en los distintos tópicos.

La tarea **task_input** se encargara de recoger toda la información de los sensores, y el estado del led, formatearlos en el siguiente orden ***estado_led:valor_adc:valor_tmp:valor_hum*** esa información formateada se enviara hacia la segunda tarea que es la encargada de publicar la información en el bróker. Esto se hace por medio de una cola. Los datos deben de ser parseados, separando los valores.

```

void task_input(void *params){
    while(1){
        int current_level = gpio_get_level(PIN_SEND);

        if(current_level == 1 && press_handled_send == 0){
            //pequeño antirebote para verificar que sea un shortpress
            vTaskDelay(pdMS_TO_TICKS(50));

            //si es así entonces procede a recibir los datos para poder enviarlos
            if(gpio_get_level(PIN_SEND) == 1){
                char value_adc[6];
                char current_state_led[2];
                char temp[10];
                char hum[10];

                //traemos un valor de ADC
                int read_adc = read_adc(ADC_CHANNEL);
                //el led solo va a cambiar cuando se publique en el tópico en el que este led esta suscrito
                //Lectura del sensor

                if(!dht11_read(&dht11_sensor, CONFIG_CONNECTION_TIMEOUT)){
                    snprintf(temp, sizeof(temp), "%0.2f", dht11_sensor.temperature);
                    snprintf(hum, sizeof(hum), "%0.2f", dht11_sensor.humidity);
                }
                else{
                    // Lectura fallida: poner valores predeterminados
                    snprintf(temp, sizeof(temp), "0.0");
                    snprintf(hum, sizeof(hum), "0.0");
                }
                snprintf(value_adc, sizeof(value_adc), "%d", read_adc);
                snprintf(current_state_led, sizeof(current_state_led), "%d", led_state);

                ESP_LOGI(TAG, "DEBUG : STATE-LED: %s", current_state_led);
                ESP_LOGI(TAG, "DEBUG : value ADC : %s", value_adc);
                ESP_LOGI(TAG, "DEBUG : Temp: %s", temp);
                ESP_LOGI(TAG, "DEBUG : Humedad: %s", hum);

                char format_values[BUFFER];

                snprintf(format_values, sizeof(format_values), "%s:%s:%s:%s", current_state_led, value_adc, temp, hum);

                xQueueSend(data_queue, &format_values, portMAX_DELAY);
                press_handled_send = 1;
            }
        }
        // cuando sueltas el botón reseteamos la bandera
        if(current_level == 0){
            press_handled_send = 0;
        }

        vTaskDelay(pdMS_TO_TICKS(10));
    }
}

```

Tarea **send_task()** es la tarea encargada de realizar el envío de información, recibe de la cola toda la cadena de texto pero debe de separar, para eso se utiliza la función **parser_input** que devuelve un arreglo con todos los tokens introducidos.

```
void send_task(void *params){

    char buffer[BUFFER];

    while(1){
        if(xQueueReceive(data_queue, (void*)buffer, portMAX_DELAY)){

            char **tokens = pasrse_input(buffer);

            //perimot Lo que ingresa lo primero en el string es LED:ADC:HUM:TMP

            esp_mqtt_client_publish(client, TOPIC_LED, tokens[0], 0, 1, 0);
            esp_mqtt_client_publish(client, TOPIC_ADC, tokens[1], 0, 1, 0);
            esp_mqtt_client_publish(client, TOPIC_TEMP, tokens[2], 0, 1, 0);
            esp_mqtt_client_publish(client, TOPIC_HUM, tokens[3], 0, 1, 0);

        }
    }

}
```

```
char **pasrse_input(char *line){

    char **tokens = malloc(5 * sizeof(char*));
    char *token;
    int position=0;

    token = strtok(line, ":");

    while(token != NULL){

        if(position >= 5 ) break;

        tokens[position++] = strdup(token);

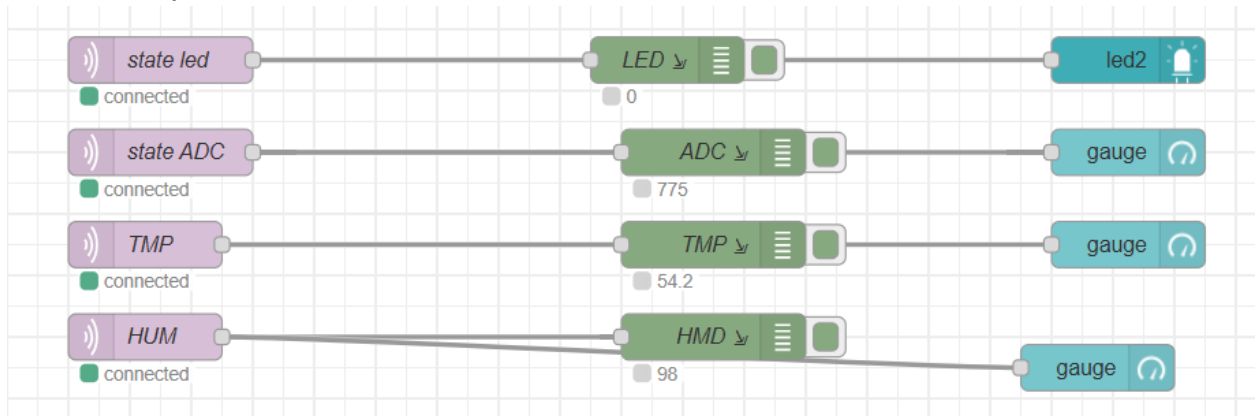
        token = strtok(NULL, ":");
    }

    tokens[position] = NULL;

    return tokens;

}
```

Cunado se publica lo recibe **node-red**.



Así como un debug que se integró en el código

```
I (6527) MQTT LIB: MQTT_EVENT_SUBSCRIBED msg_id=23341
I (13887) <MAIN> NODE-RED_MQTT > : DEBUG : STATE-LED: 0
I (13887) <MAIN> NODE-RED_MQTT > : DEBUG: value ADC : 775
I (13887) <MAIN> NODE-RED_MQTT > : DEBUG : Temp: 54.20
I (13887) <MAIN> NODE-RED_MQTT > : DEBUG: Humedad: 98.00
```

Podemos verificar que en efectos los datos son iguales.

La tarea **task_input_topic** es la encargada de esperar lo que se recibe de las publicaciones suscritos y en este caso está esperando que el **payload** de la publicación sea **ON / OFF** para encender o apagar el led.

```
void task_input_topic(void *params){
    char *action;

    while(1){
        if(xQueueReceive(flow_data, &action, portMAX_DELAY)){
            char **tokens = parse_input(action); //separamos en sus diferentes tokens

            ESP_LOGI(TAG, "topic: token[0] = [%s]", tokens[0]);
            ESP_LOGI(TAG, "payload: token[1] = [%s]", tokens[1]);

            if(!strcmp("ON", tokens[1])){
                gpio_set_level(PIN_OUT, 1);
                led_state = 1;
            }
            else if(!strcmp("OFF", tokens[1])){
                gpio_set_level(PIN_OUT, 0);
                led_state=0;
            }
        }

        //Liberandi memoria
        for(int i = 0; tokens[i] != NULL; i++){
            free(tokens[i]);
        }
        free(tokens);
        // Liberar el mensaje recibido
        free(action);
    }

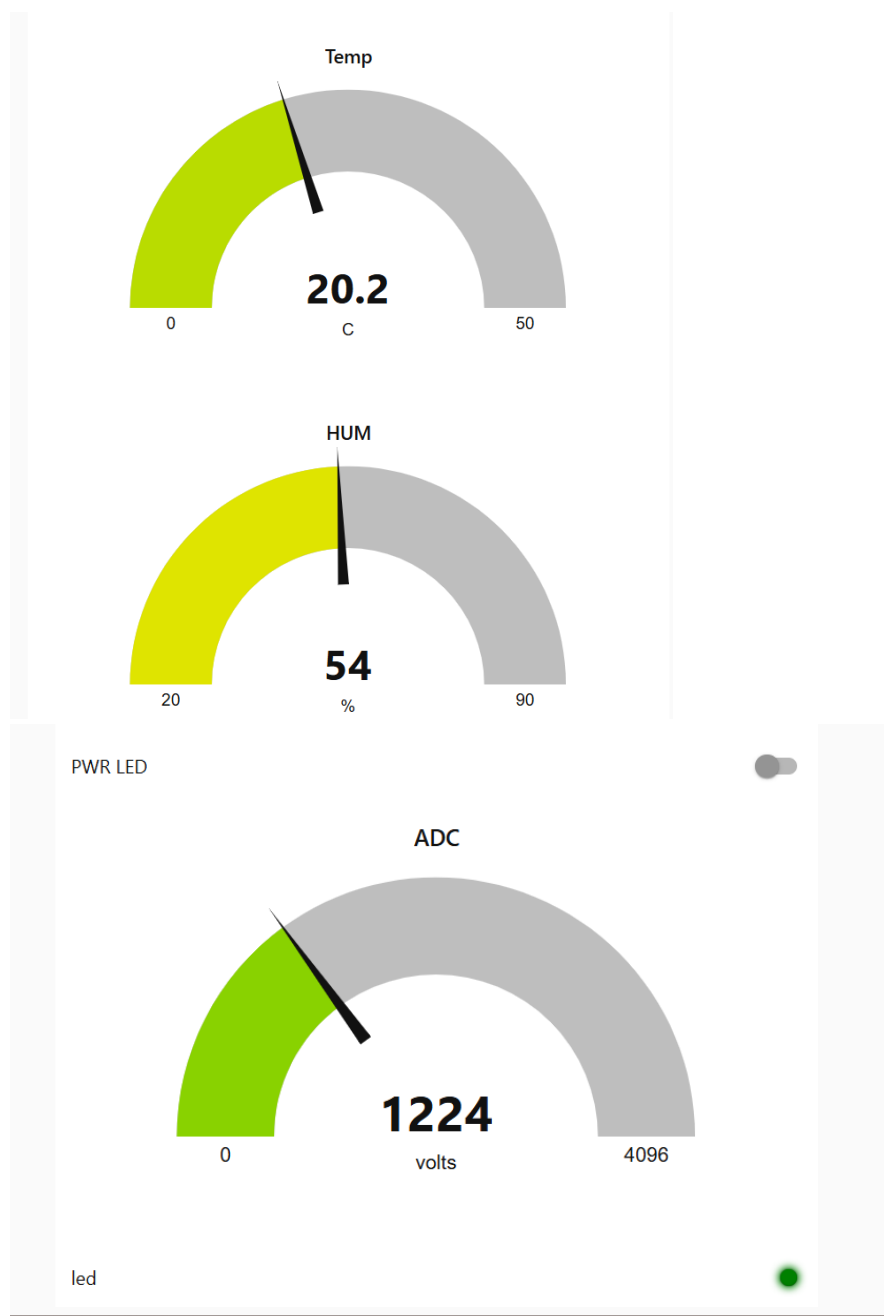
    vTaskDelay(pdMS_TO_TICKS(10));
}
}
```

La otra forma de prender el led es mediante una tarea, porque se eligió la tarea, porque si se realizaba mediante una función en algún punto de alguna tarea el cambio no se podría realizar ya que se estaría bloqueando, o se tendría que llamar en el momento preciso durante la ejecución de la tarea.

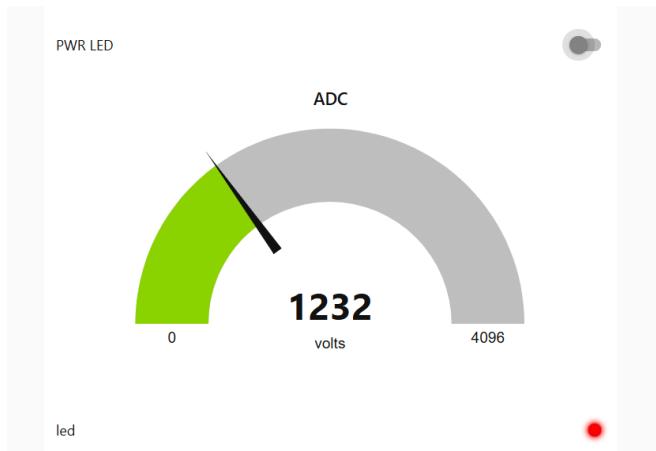
Por lo que una tarea siempre estará esperando la entrada por GPIO. Por lo que no depende de otras tareas es independiente.

```
void gpio_task(void *params){  
  
    while(1){  
        int current_level = gpio_get_level(PIN_TOGG_LED);  
  
        if(current_level == 1 && press_handled_led == 0){  
  
            //pequeno antirebote para verificar que sea un shortpress  
            vTaskDelay(pdMS_TO_TICKS(50));  
  
            //si es asi entonces procede a recopilar los datos para poder unirlos  
            if(gpio_get_level(PIN_TOGG_LED) == 1){  
                led_state = !led_state;  
                gpio_set_level(PIN_OUT, led_state);  
                press_handled_led = 1;  
            }  
        }  
  
        // cuando sueltas el botón reseteamos la bandera  
        if(current_level == 0){  
            press_handled_led = 0;  
        }  
        vTaskDelay(pdMS_TO_TICKS(10));  
    }  
}
```

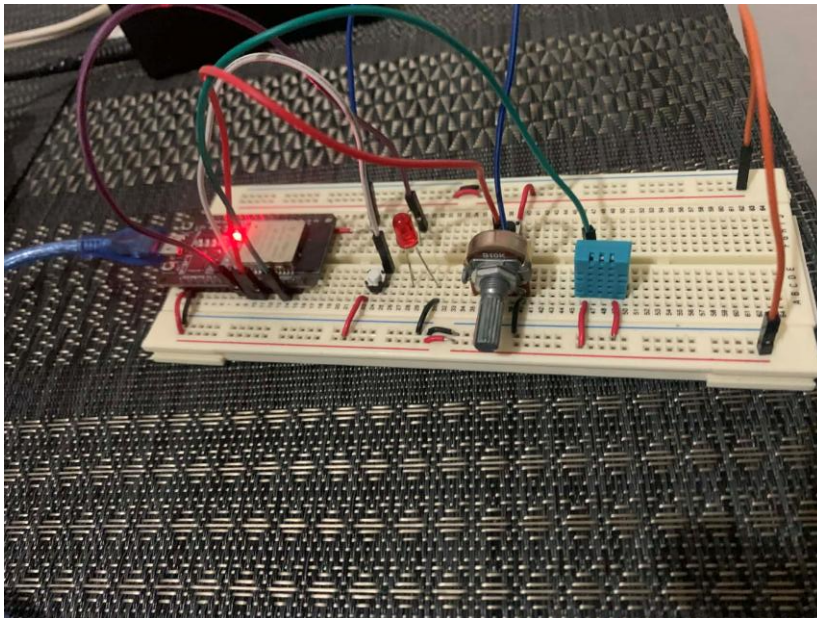
La vista para el usuario en un dashboard, sería la siguiente, tendrá 3 graficas en donde se mostrará la intensidad de los distintos componentes.

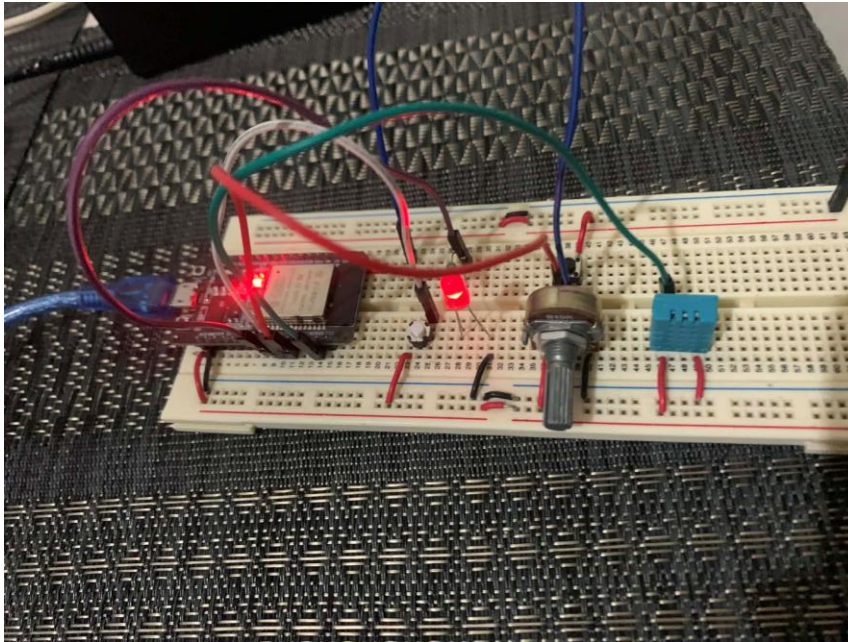


El switch **PWR LED** funciona para publicar en el bróker que queremos apagar o prender el led y el punto verde indica el estado del led cuando se publica su estado mediante el botón físico.



3. Hardware





4. Dificultades

Para el Desarrollo de la práctica no se presentaron dificultades, al inicio fue el pensar y plantear como se iban a pasar todos los datos de los distintos dispositivos. Pero fuera de eso la estructura es muy parecido a prácticas con MQTT anteriores ya vistas.

5. Conclusiones

Durante la práctica se pudo utilizar una herramienta muy útil como lo es node-red, una herramienta visual y de low-code con la que se pueden hacer cosas muy impresionantes como lo puede ser el crear dashboards con la información de los componentes conectados a nuestro ESP32, esto puede ser una vista muy simple en lo que pudiera ser la construcción de dispositivos IoT, con un una interfaz más intuitiva y simple, lo que cualquiera pudiera entender, se puede ver la versatilidad de la comunicación MQTT.